

Guía Introductoria al Lenguaje Perl

versión 3.0

Fernando Bordignon, Gabriel Tolosa, Fernando Lorge y Josafa Pontes

{bordi, tolosoft, florge}@mail.unlu.edu.ar, josafa.aguiar@epn.edu.ec

La presente guía es una versión "quick and dirty" -tal como dicen los anglosajones- con el objetivo de lograr un primer acercamiento al lenguaje. Se sugiere complementar el estudio con bibliografía adicional, como la recomendada.

Charles Babbage dijo *"se cometen muchos menos errores usando datos incorrectos, que no empleando dato alguno"*. Es por esto que se pide perdón por los posibles errores que se hayan deslizado, y se solicita al lector que contribuya a mejorar este material de estudio, a partir de las correcciones y críticas que pueda aportar.

Fuentes de bibliografía adicional sobre el lenguaje Perl:

- Página oficial del lenguaje Perl. <http://www.perl.com>
- Curso de Perl (en inglés) <http://fpg.uwaterloo.ca/perl/start.html>
- Perl Mongers. <http://www.perl.org/>
- Editorial O'Reilly, página del lenguaje Perl. <http://perl.oreilly.com/>
- PerlDoc. <http://www.perldoc.com/>
- The Perl Journal. <http://www.tpj.com/>
- Perl Chile. <http://www.perl.cl/>
- En su unix escriba: man perl

Lugares de descarga del lenguaje:

- Página oficial del lenguaje Perl. <http://www.perl.com>
- CPAN - Módulos Perl. <http://www.cpan.org/>
- Activestate. Perl para Windows. <http://www.activestate.com/>

Filosofía de Perl:

"Hay más de una forma de hacerlo" según Larry Wall, su autor.

Introducción al Lenguaje

"Si no noto la resistencia del papel frente a la pluma, me siento incapacitado" Jean Daniel

Perl significa Lenguaje práctico de extracción y reporte (Practical Extraction and Report Language). Su creador fue Larry Wall, y su objetivo era simplificar las tareas habituales a realizar en el sistema operativo

Unix. Hoy es un lenguaje de propósito general, de alta portabilidad y es la herramienta principal que utilizan los webmasters para implementar programación a través de la interfase CGI.

Perl es un lenguaje interpretado, aunque transparentemente compile los programas antes de ejecutarlos. Es por ello que los ortodoxos de la comunidad hablan de guiones (scripts) y no de programas. Existe una numerosa cantidad de programadores Perl distribuidos a lo largo del mundo, ellos día a día producen nuevos módulos de librería que extienden asombrosamente las capacidades del lenguaje. En Perl, prácticamente, es posible programar casi cualquier problema; maneja: comunicaciones entre computadoras, bajo protocolos TCP/IP; implementa hilos de ejecución (threads); implementa - opcionalmente- el estilo de programación orientado a objetos, posee APIs con todos los protocolos populares de aplicación en Internet. Aunque su punto debil es la performance en la ejecución de guiones - no alcanza las velocidades ofrecidas por lenguajes como el C-, Perl es un excelente lenguaje destinado al aprendizaje y el desarrollo de prototipos de modelos de software.

PERL es gratuito, hay completa libertad de copiar el código fuente, compilarlo, imprimirlo, compartirlo, etc, pero todo esto es bajo la licencia GNU, que en términos generales ordena que cualquier programa que se desarrolle se debe liberar, o sea si Ud. puede utilizar el código hecho por otros; otros pueden usar el código hecho por Ud.

Como se mencionó anteriormente, la portabilidad es una características de Perl. Los guiones se pueden ejecutar sobre distintos sistemas operativos como ser: Linux, System de Macintosh, Microsoft Windows, Solaris de Sun, BSD, etc. En definitiva, un programa puede correr en cualquier sistema operativo sin tener que modificar el código fuente; solamente es necesario poseer el interprete de Perl para cada sistema y librerías adicionales que use.

Con Perl se pueden: construir pequeños programas destinados a ser usados como filtros para obtener información en base a la lectura de archivos, realizar búsquedas, etc. Es uno de los lenguajes más utilizados en la programación de la interfase CGI, que sirve para el intercambio de información entre aplicaciones externas y el espacio web. Por ejemplo programas de búsqueda, y procesamiento de formularios, entre otros.

El siguiente es el clásico ejemplo de primer programa Perl

```
In [1]: # Todo código debe tener al menos esas 3 funciones
use strict; # Forza la declaración de las variables
use warnings; # Genera mensajes de error de sintaxis
use Data::Dump qw(dump); # Para la impresión de estructuras de datos
```

```
In [2]: # Librerías adicionales
use List::Util qw(zip min max sum any all first none); # Reorganizar los arreglos con zip
use Tie::IxHash; # Preserva el orden de registro en arreglos asociativos
# use aliased 'jjap::numperl' => 'np';
```

```
In [3]: print "Hola Mundo!!!\n";
```

Hola Mundo!!!

```
Out[3]: 1
```

En la primer linea se le indica al sistema operativo el camino del interprete Perl, para que lo pueda cargar y ejecutar, y le pase como parámetro el guión que se halla más abajo. Por norma, todo guión lleva la extensión pl y se lo escribe utilizando un editor ASCII normal. Inicialmente, para ejecutar un programa siga la siguiente sintaxis:

```
perl <nombre_del_programa.pl>
```

Para un mejor análisis de un programa, puede redireccionarse su salida a un archivo utilizando en la orden de ejecución el símbolo de redirección ">" y a continuación un nombre de archivo.

Tipos de Datos, Variables y Arreglos

"La comunicación se compone de quien habla, quien escucha y lo que se dice". Aristóteles

Para un mejor análisis de un programa, puede redireccionarse su salida a un archivo utilizando en la orden de ejecución el símbolo de redirección ">" y a continuación un nombre de archivo.

Perl permite representar los siguientes tipos de datos básicos: reales, enteros, cadenas de caracteres y booleanos.

Los tipos numéricos (reales y enteros).

Los valores numéricos se presentan en forma de valores reales codificados en doble precisión. Este formato interno se utiliza para todas las operaciones aritméticas.

```
In [4]: my $z = 0.127; # real
        my $x = 3.22e-14; # real
        my $c = 1567; # entero
        my $d = -122; # entero
        print $x;
```

3.22e-14

Out[4]: 1

Los valores enteros no pueden empezar por cero porque esto permite especificar un entero mediante su codificación octal o hexadecimal. El código octal comienza con cero 0; el código hexadecimal comienza con 0x.

```
In [5]: $x = 0377; # Representación octal, equivale a 255 decimal
        my $y = 0xff; # Representación hexadecimal, equivale a 255
```

Out[5]: 255

```
In [6]: print $x;
```

255

Out[6]: 1

```
In [7]: print sprintf("%o", $x);
```

377

Out[7]: 1

```
In [8]: print $y;
```

255

Out[8]: 1

```
In [9]: print sprintf("%X", $y);
```

FF

Out[9]: 1

```
In [10]: print sprintf("%.3f", 3.14151692);
```

3.142

Out[10]: 1

Las cadenas de caracteres.

Las cadenas de caracteres se representan por medio de una sucesión de caracteres delimitada por comillas ó apóstrofes. Cuando una cadena se delimita por comillas, toda variable referenciada en el interior de la cadena se evalúa y se reemplaza por su valor. Por ejemplo:

```
In [11]: my $cadena = "Brothers\t$x\n";  
print $cadena;
```

Brothers 255

Out[11]: 1

```
In [12]: $cadena = 'Brothers\t$x\n';  
print $cadena;
```

Brothers\t\$x\n

Out[12]: 1

Por el contrario, las cadenas de caracteres delimitadas por apóstrofes se dejan intactas.

Existe una sintaxis especial que permite delimitar una cadena de caracteres cuando contiene varias líneas y/o comillas o apóstrofes. El fin de la cadena se determina por la nueva línea que contiene únicamente el identificador. Éste no debe ir precedido por un espacio ni marca de tabulación. Por ejemplo:

```
In [13]: $cadena = <<SALUDO;  
hola,  
buenos días,  
adios,  
SALUDO
```

Out[13]: hola,
buenos días,
adios,

El tipo booleano.

El tipo booleano existe de modo implícito, es decir, un número es falso si es igual a cero y verdadero en cualquier otro caso. Como el cero está asociado a la cadena vacía (""), ésta también equivale al valor falso.

```
In [14]: $x = 0;
         if ($x){
           print "Verdadero";
         }else{
           print "Falso";
         }

```

Falso

Out[14]: 1

```
In [15]: $x = "";
         if ($x){
           print "Verdadero";
         }else{
           print "Falso";
         }

```

Falso

Out[15]: 1

```
In [16]: my $p;
         if ($p){
           print "Verdadero";
         }else{
           print "Falso";
         }

```

Falso

Out[16]: 1

```
In [19]: print dump $p;

```

undef

Out[19]: 1

```
In [17]: print $p; # Muestra una advertencia, ya que $p vale undef.

```

Out[17]: 1

Use of uninitialized value \$p in print at reply input line 1.

Representaciones de datos.

Perl maneja datos escalares, siendo todos aquellos que contienen un valor; no importa si es real, entero, caracter o cadena; el lenguaje los reconoce automáticamente. El lenguaje posee tres tipos de representaciones de datos

Variables escalares.

Arreglos indexados escalares (listas).

Arreglos asociativos escalares (hash).

Las representaciones de datos son determinadas por la anteposición de un caracter especial.

\$ Escalar.

@ Arreglo Indexado.

% Arreglo Asociativo.

Todas las representaciones tienen su propio nombre, así como las etiquetas, funciones, archivos y manejadores de directorios.

```
In [20]: my @array = (); # Declaro arreglo e lo inicializo vacío
print dump @array;
```

```
()
Out[20]: 1
```

```
In [21]: print "@array";
```

```
Out[21]: 1
```

```
In [22]: @array = (10, 3, 7, "word");
print dump @array;
```

```
(10, 3, 7, "word")
Out[22]: 1
```

```
In [23]: push @array, "new"; # Agrega al final del array
print "@array";
```

```
10 3 7 word new
Out[23]: 1
```

```
In [24]: unshift @array, "beginning"; # Agrega al inicio del array
print dump @array;
```

```
("beginning", 10, 3, 7, "word", "new")
Out[24]: 1
```

```
In [25]: splice @array, 2, 0, "between"; # Agrega en una posición arbitraria
print dump @array;
```

```
("beginning", 10, "between", 3, 7, "word", "new")
Out[25]: 1
```

```
In [26]: print $array[3];# Imprimir el 3
```

```
3
Out[26]: 1
```

```
In [28]: print $#array;
```

```
6
Out[28]: 1
```

```
In [29]: print dump @array;
```

```
("beginning", 10, "between", 3, 7, "word", "new")
Out[29]: 1
```

```
In [30]: print $#array;
```

```
6
```

Out[30]: 1

```
In [31]: my @array2 = @array[1 .. $#array]; # Imprimir 3, 7, "word", necesito un slice
print "@array2";
```

10 between 3 7 word new

Out[31]: 1

```
In [32]: my $cant = scalar(@array2);
```

Out[32]: 6

```
In [33]: print dump @array[1, 3, 5]; # Slice de posiciones impares
```

(10, 3, "word")

Out[33]: 1

```
In [34]: my $var1 = pop @array;# Retiro el elemento final del arreglo
print "var1:$var1\n";
print "array:", dump @array;
```

var1:new

array:("beginning", 10, "between", 3, 7, "word")

Out[34]: 1

```
In [35]: $var1 = shift @array;# Retiro el elemento inicio del arreglo
print "var1:$var1\n";
print "array:", dump @array;
```

var1:beginning

array:(10, "between", 3, 7, "word")

Out[35]: 1

```
In [36]: @array = (1 .. 5);
$var1 = splice @array, 2, 0, ( 6 .. 9 );# Retiro un elemento de una posición arbitraria
print "var1:$var1\n" if defined $var1;
print "array:", dump @array;
```

array:(1, 2, 6 .. 9, 3, 4, 5)

Out[36]: 1

```
In [37]: my @array2 = @array; # Copia simple
$array2 = ();
print dump @array;
```

(1, 2, 6 .. 9, 3, 4, 5)

Out[37]: 1

```
In [38]: my $array2 = \@array; # Referencia - afecta los cambios de $array2.
@$array2 = ();
print dump @array;
```

()

Out[38]: 1

```
In [39]: @array = (1 .. 5);
push @array, @array; # Concatenar 2 listas
print "array:", dump @array;
```

array:(1 .. 5, 1 .. 5)

Out[39]: 1

```
In [40]: my @list1 = (0, 1, 2);  
my @list2 = (3, 4, 5);  
# Producir una lista3 que contenga los valores alternados  
# de las 2 listas dadas: (0, 3, 1, 4, 2, 5) en un solo comando.
```

Out[40]: 345

```
In [41]: my @lista3 = (@list1, @list2); # Concatenación de 2 listas simples  
print "@lista3"; # Impresión de los miembros de la lista
```

0 1 2 3 4 5

Out[41]: 1

```
In [42]: print dump @lista3; # Impresión de la lista como tal
```

(0 .. 5)

Out[42]: 1

```
In [43]: print \@lista3; # Impresión de la referencia de una lista: la dirección de memoria es la  
# La referencia de una variable se obtiene a través de una barra invertida
```

ARRAY(0x57f2858)

Out[43]: 1

```
In [44]: print dump zip (\@list1, \@list2); # La función zip agrupa los elementos de cada lista a  
# Las entradas son referencias a arreglos.  
# Produce una lista simple como salida.
```

([0, 3], [1, 4], [2, 5])

Out[44]: 1

```
In [45]: # El contenido entre paréntesis conforma una lista simple,  
# representada por arroba @. () se asocia con @ - lista simple.  
# Para convertir una lista simple @ en referencia: \@  
  
# El contenido entre corchetes conforma una referencia a un arreglo,  
# representada por dólar $. [] se asocia con $ - referencia a arreglo.  
# Para convertir una referencia $ a una lista simple: @$ o @{}
```

```
In [46]: print dump zip ([0, 2, 4], [1, 3, 5]); # Los corchetes también son referencias a listas.
```

([0, 1], [2, 3], [4, 5])

Out[46]: 1

```
In [47]: my $x = [0, 2, 4]; # La variable x guarda la referencia a una lista.  
print $x;
```

ARRAY(0x5802268)

Out[47]: 1

```
In [48]: print "@$x"; # Desreferenciación de $x
```

0 2 4

Out[48]: 1

```
In [49]: my $y = [1, 3, 5];  
print dump zip ($x, $y); # Referencias a listas pueden ser variables.
```



```

([0, 1], [2, 3], [4, 5])
Out[49]: 1

In [50]: my @list3 = map {$_->[0] + $_->[1]} zip ($x, $y); # La función map desarma las agrupaciones
print dump @list3;

(1, 5, 9)
Out[50]: 1

In [51]: print dump none {$_ % 3 == 0} @list3;

""
Out[51]: 1

```

Variables escalares.

Se utiliza el caracter \$ para indicar un valor escalar. La asociación ó bind de una variable con valor se hace con el caracter =.

```

my $foo = 55.23;
$foo = 'esta es una cadena';
$foo = "era $foo antes";      # variable interpolada.
my $salida= `cd`;             # Se invoca al sistema operativo y el
resultado de la salida        # se almacena en el escalar de la
                               # izquierda.
($a, $b) = ($b, $a);          # intercambio (swap)

```

Ejemplo:

Sumar dos números y luego aplicar el operador de post-incremento

```

In [52]: printf "lista1:%s\n", dump @list1;
my ($A, $B) = @list1;
my $suma = $A + $B;
print "Resultado:" . $suma . "\n";
$suma++;
print "Despues de incrementar: $suma \n";
print $suma += 3;

lista1:(0, 1, 2)
Resultado:1
Despues de incrementar: 2
5
Out[52]: 1

```

Ejemplo:

Verificar el método de intercambio de variables

```

In [53]: $A = 'Mundo';
$B = 'Hola';
($A, $B) = ($B, $A); # intercambio de variables
print "La famosa frase es $A $B!!! \n";

La famosa frase es Hola Mundo!!!
Out[53]: 1

```

Las secuencias de escape se utilizan a los efectos de representar caracteres especiales ó modificar un valor. Algunas son

\n	Nueva línea	
\t	Tabulador	Print "dato \t separado\n";
\oo	oo valor octal de ASCII	
\xhh	hh valor hexadecimal de ASCII	
\\	Diagonal invertida	Print "la barra \ sirve ...";
\"	Comillas dobles	Print "las comillas \" sirven ...";
\L	Minúsculas todas las letras hasta \E	Print "\LEN MINUSCULAS\E";
\U	Mayúsculas todas hasta \E	Print "\Uen mayusculas\E"; "\Q \E"
\Q	Inicia escape de caracteres especiales	
\E	Terminar \L, \U, \Q	

Ejemplo:

Ingresar dos cadenas y comparar si son iguales, independientemente que sus caracteres esten en mayúsculas o minúsculas.

```
In [54]: my $cadena = "palabraA\n"; # Se almacena una cadena con \n
chomp($cadena); #chomp elimina el \n de la cadena
print "$cadena ya no tiene salto de fila.";
```

palabraA ya no tiene salto de fila.

Out[54]: 1

```
In [55]: print "\U$cadena\E convertida a mayúscula.";
```

PALABRAA convertida a mayúscula.

Out[55]: 1

```
In [56]: print "\L$cadena\E convertida a minúscula.";
```

palabraa convertida a minúscula.

Out[56]: 1

Arreglos

Arreglos Indexados

En PERL los arreglos son listas de datos sin importar su tipo, es decir que puede estar compuestos por enteros flotantes, cadenas, o incluso otros arreglos, cada elemento es considerado como una variable independiente. El caracter \$ se utiliza para referirse a un elemento en particular y el caracter @ para todos. El primer elemento de un arreglo es el indicado con 0 (cero).

```
In [57]: my @arreglo = (); # Creamos una lista vacía
print dump @arreglo;
```

()

Out[57]: 1

```
In [58]: push @arreglo, 3; # Le agregamos un valor al final de la lista mediante push.  
print "@arreglo";
```

3
Out[58]: 1

```
In [59]: push @arreglo, 4; # Otro elemento más mediante push.  
print "@arreglo";
```

3 4
Out[59]: 1

```
In [60]: # Use el caracter @ para referirse a todos los miembros del arreglo indexado  
print dump @arreglo;
```

(3, 4)
Out[60]: 1

```
In [61]: # Use el caracter $ para referirse a un miembro específico del arreglo indexado  
print $arreglo[0];
```

3
Out[61]: 1

```
In [62]: $arreglo[0] = 5; # Cambiamos el valor de ese elemento  
print $arreglo[0];
```

5
Out[62]: 1

Copiar un arreglo hacia otro

```
In [63]: my @arreglo = (1, 3, 5); # Inicializamos el arreglo  
my @foo = @arreglo; # Copiamos el arreglo  
@arreglo = (); # Limpiamos el arreglo  
print "arreglo:", dump (@arreglo), "\n";  
print "foo:", dump (@foo), "\n";
```

arreglo:()
foo:(1, 3, 5)
Out[63]: 1

```
In [64]: print dump @foo[0, 2]; # Slice del arreglo
```

(1, 5)
Out[64]: 1

```
In [65]: # El operador .. genera un rango de valores enteros en una lista.  
print dump (-1 .. 3), "\n";  
print dump (-2 .. -2), "\n"; # Note que incluye el último elemento.
```

(-1 .. 3)
-2
Out[65]: 1

```
In [66]: print dump (reverse -1 .. 3);
```

(3, 2, 1, 0, -1)

Out[66]: 1

```
In [67]: print dump (map { $_ / 10 } -1 .. 3);
```

(-0.1, 0, 0.1, 0.2, 0.3)

Out[67]: 1

```
In [68]: print dump (grep { $_ > 0 } -1 .. 3);
```

(1, 2, 3)

Out[68]: 1

```
In [69]: printf "Índice del último elemento de foo: %d\n", $#foo; # $#foo es el índice del último  
printf "Slice de foo: %s\n", dump @foo[0 .. $#foo]; # Slice del arreglo tomando todos los
```

Índice del último elemento de foo: 2

Slice de foo: (1, 3, 5)

Out[69]: 1

```
In [70]: print $foo[-1], "\n"; # Devuelve el último elemento.  
print $foo[-2], "\n"; # Devuelve el penúltimo elemento.
```

5

3

Out[70]: 1

```
In [71]: my @A = (95, 7, 'fff' ); # se instancia el arreglo a con 3 elementos  
printf "%s\n", $A[-1]; # imprime el último elemento: fff  
print "@A"; # imprime los elementos de la lista
```

fff

95 7 fff

Out[71]: 1

Ejemplo:

```
In [72]: my @arreglo1 = (10, 20, 30);  
my @arreglo2 = (100, 200);  
my @arreglo3 = (@arreglo1, @arreglo2, 8, "es una cadena");  
# Nota: Se a inicializado arreglo 3 con dos arreglos y dos  
# variables escalares, por lo cual arreglo 3 pasó  
# a tener 7 elementos.  
my $len_arreglo3 = scalar(@arreglo3);  
print "Longitud del arreglo3: ", $len_arreglo3, "\n";  
print "Posición del último elemento de arreglo3: ", $#arreglo3, "\n";  
print "Longitud del arreglo3: ", scalar (@arreglo3), "\n";
```

Longitud del arreglo3: 7

Posición del último elemento de arreglo3: 6

Longitud del arreglo3: 7

Out[72]: 1

```
In [73]: # imprimir el arreglo  
for (my $i = 0; $i < @arreglo3; $i++) {  
    printf "%d\t%s\n", $i, $arreglo3[$i];  
}
```

```
0      10
1      20
2      30
3      100
4      200
5      8
6      es una cadena
```

```
In [74]: print "$_\n" for @arreglo3;
```

```
10
20
30
100
200
8
es una cadena
```

```
In [75]: for (@arreglo3){
         printf "%s\n", $_;
       }
```

```
10
20
30
100
200
8
es una cadena
```

```
In [76]: for my $elem (@arreglo3){
         printf "%s\n", $elem;
       }
```

```
10
20
30
100
200
8
es una cadena
```

```
In [77]: foreach my $elem (@arreglo3){
         printf "%s\n", $elem;
       }
```

```
10
20
30
100
200
8
es una cadena
```

```
In [78]: while (my ($i, $elem) = each @arreglo3){
         printf "%d\t%s\n", $i, $elem;
       }
```

```
0      10
1      20
2      30
3      100
4      200
5      8
6      es una cadena
```

Out[78]: 0

Se pueden definir valores de una lista por medio de la técnica de rangos usando el operador ..

```
In [79]: my @A = (2 .. 7); # $a se instancia con (2,3,4,5,6,7);
my @B = ('a' .. 'e'); # $b instancia con ('a','b','c','d','e')
```

```
print "$_ " for @A;
print "\n";
print "$_ " for @B;
```

```
2 3 4 5 6 7
a b c d e
```

```
In [80]: print 'Suma de @A: ', sum(@A);
```

Suma de @A: 27

Out[80]: 1

La función **join** convierte un arreglo en un escalar y la función **split** hace lo inverso, convierte una variable escalar en una lista.

```
In [81]: @A = ('a' .. 'e');
$A = join ":", @A;
print $A; # se obtiene la cadena "a:b:c:d:e"
```

a:b:c:d:e

Out[81]: 1

```
In [82]: my @lista = split /[A-Z]+/, "Estales2877una3frase";
print dump @lista;
```

("", "stales2877una3frase")

Out[82]: 1

```
In [83]: my $str = "Estales2877una3fraseEscuelaPolitecnicaNacional";
my $regex = '[A-Z][a-z]*';

for my $word ($str =~ m/$regex/g){
    printf "%s ", $word;
}
```

Esta Escuela Politecnica Nacional

Splice:

La función splice permite extraer un subarreglo y a la vez modificar el arreglo original

```
In [85]: @A = ('a' .. 'e'); # Genero un array
@B = splice(@A, 1, 2); # Retiro los a partir de la posicion 1 dos elementos y los guardo
printf "@A: %s\n", "@A";
printf "@B: %s\n", "@B";
```

```
@A: a d e
@B: b c
```

Out[85]: 1

```
In [87]: @A = ('a' .. 'e');
@B = (1 .. 3);
splice @A, 2, 1, @B;
```

```
print '@A: ', dump (@A), "\n";

# La 'c' se reemplazó con (1, 2, 3)
```

```
@A: ("a", "b", 1, 2, 3, "d", "e")
```

Out[87]: 1

In [88]: *# Es posible insertar sin eliminar elemento alguno*

```
@A = ( 'a' .. 'e' );
@B = ( 1 .. 3 );
splice( @A, 2, 0, @B );
print '@a: ', dump (@A), "\n";
# Se imprime ("a", "b", 1, 2, 3, "c", "d", "e");
```

```
@a: ("a", "b", 1, 2, 3, "c", "d", "e")
```

Out[88]: 1

Referencia a arreglos multi-dimensionales

En la siguiente instrucción se define una referencia a un arreglo de 4 filas por 3 columnas

In [89]:

```
my $arreglo = [ [1, 2, 3], [4, -5, 6], [7, 8, 9], [10, 11, 12] ];
print dump $arreglo;
```

```
[[1, 2, 3], [4, -5, 6], [7, 8, 9], [10, 11, 12]]
```

Out[89]: 1

In [90]:

```
print $arreglo; # Es una referencia
```

```
ARRAY(0x5826828)
```

Out[90]: 1

In [91]:

```
print dump $arreglo->[3] # Uso la FLECHA para obtener una fila, puesto que la referencia
```

```
[10, 11, 12]
```

Out[91]: 1

In [92]:

```
print $arreglo->[3][1]; # Uso la flecha para obtener un elemento de la referencia $arreg
```

```
11
```

Out[92]: 1

In [93]:

```
print dump @$arreglo[2]; # Mediante la desreferenciación del array (símbolo @ antes), ob
```

```
[7, 8, 9]
```

Out[93]: 1

In [94]:

```
print dump $arreglo->[3] # Desreferenciación de la cuarta fila
```

```
[10, 11, 12]
```

Out[94]: 1

In [95]:

```
print dump @{$arreglo->[3]}[0, 2] # Obtengo un slice de la desreferenciación de la cuart
```

```
(10, 12)
```

Out[95]: 1

In [96]:

```
print dump @$arreglo[1, 3]; # Un slice de la segunda y cuarta filas
```

```
([4, -5, 6], [10, 11, 12])
```

Out[96]: 1

```
In [97]: my $longitud = @$arreglo;
print "# filas: ", $longitud, "\n"; # Longitud del arreglo: número de filas
my $columnas = @{$arreglo->[0]}; # Desreferenciación de la fila 0
print "# columnas: ", $columnas, "\n";
```

```
# filas: 4
# columnas: 3
```

Out[97]: 1

```
In [98]: for (my $i = 0; $i < @$arreglo; $i++){ # Barrido de filas.
print dump ($arreglo->[$i]), "\n";
}
```

```
[1, 2, 3]
[4, -5, 6]
[7, 8, 9]
[10, 11, 12]
```

```
In [99]: for (@$arreglo){ # Barrido de filas.
print dump ($_), "\n";
}
```

```
[1, 2, 3]
[4, -5, 6]
[7, 8, 9]
[10, 11, 12]
```

```
In [100]: print dump ($_), "\n" for @$arreglo; # Barrido de filas en una linea de código
```

```
[1, 2, 3]
[4, -5, 6]
[7, 8, 9]
[10, 11, 12]
```

```
In [101]: for (my $i = 0; $i < @$arreglo; $i++){ # Barrido de filas.
for (my $j = 0; $j < @{$arreglo->[$i]}; $j++){ # Barrido de columnas.
print $arreglo->[$i][$j], "\t";
}
print "\n";
}
```

```
1      2      3
4      -5     6
7      8      9
10     11     12
```

```
In [102]: for my $fila (@$arreglo){ # Barrido de filas.
for my $cell (@$fila){ # Barrido de columnas.
print $cell, "\t";
}
print "\n";
}
```

```
1      2      3
4      -5     6
7      8      9
10     11     12
```

```
In [103]: # También es posible crear una lista simple conformada por referencias a arreglos.
my @arreglo = ( [1, 2, 3], [4, -5, 6], [7, 8, 9], [10, 11, 12] );
```


Out[103]: ARRAY(0x582e828)ARRAY(0x5821328)ARRAY(0x582d6a8)ARRAY(0x5802190)

```
In [104... for (my $i = 0; $i < @arreglo; $i++){ # Barrido de filas.  
    print dump ($arreglo[$i]), "\n";  
}
```

```
[1, 2, 3]  
[4, -5, 6]  
[7, 8, 9]  
[10, 11, 12]
```

```
In [105... for (my $i = 0; $i < @arreglo; $i++){ # Barrido de filas.  
    for (my $j = 0; $j < @{$arreglo[$i]}; $j++){ # Barrido de columnas.  
        print $arreglo[$i][$j], "\t";  
    }  
    print "\n";  
}
```

```
1      2      3  
4      -5     6  
7      8      9  
10     11     12
```

Un arreglo no es más que una lista donde un elemento n puede ser a su vez otro arreglo, por lo tanto no es necesario que todas las filas tengan la misma longitud.

```
In [106... my @m1 = ( 1 , "maria" );  
my @m2 = ( "pablo", "guillermo", "silvina" );  
my @m3 = ( "rosa", "agustin" , 3 );  
my @m = ( @m1, @m2, @m3 ); # Concatena las listas: se desarmen las agrupaciones  
print dump @m;
```

```
(1, "maria", "pablo", "guillermo", "silvina", "rosa", "agustin", 3)
```

Out[106]: 1

```
In [107... @m = ( \@m1, \@m2, \@m3 ); # Concatena las referencias: preserva las agrupaciones  
print dump @m;
```

```
(  
  [1, "maria"],  
  ["pablo", "guillermo", "silvina"],  
  ["rosa", "agustin", 3],  
)
```

Out[107]: 1

Para determinar la cantidad de filas y columnas se utiliza:

```
In [108... print $#arreglo, "\n"; # Brinda el índice del último elemento  
print ${$arreglo[1]}, "\n"; # Índice del último elemento de la fila 1
```

```
3  
2
```

Out[108]: 1

Ejemplo:

Programa que imprime un arreglo con longitudes variables.

```
In [109... @arreglo = ( [1], [2, 3], [4 .. 6], [7, 8], [9]);
for(my $i = 0; $i <= $#arreglo; $i++) {
    for(my $j = 0; $j <= ${$arreglo[$i]}; $j++) {
        print $arreglo[$i][$j], "\t"
    };
    print "\n";
}
```

```
1
2      3
4      5      6
7      8
9
```

Un arreglo de tres dimensiones puede definirse de la siguiente manera

```
In [110... @lista = ( [[1, 2, 3], [4, 5, 6], [7, 8, 9] ],
              [["a", "b", "c"], ["d", "e", "f"], ["g", "h", "i"] ],
              [[-1, -2, -3], [-4, -5, -6], [-7, -8, -9]] );
print $lista[0][1][2], "\n"; # Imprime 6
print $lista[2][2][1], "\n"; # Imprime -8
```

```
6
-8
1
```

Out[110]:

```
In [111... my $array = [ [[1, 2, 3], [4, 5, 6], [7, 8, 9] ],
                [["a", "b", "c"], ["d", "e", "f"], ["g", "h", "i"] ],
                [[-1, -2, -3], [-4, -5, -6], [-7, -8, -9]] ];
# Requiere la flecha porque se trata de una referencia a un array, y no de una lista simple
print $array->[0][1][2], "\n";
print $array->[2][2][1], "\n";
```

```
6
-8
1
```

Out[111]:

Pilas y colas con arreglos

La función `shift` extrae el 1er elemento de un arreglo y además lo elimina. La función `pop` extrae el último elemento de un arreglo y lo elimina. Por otro lado existen las funciones `unshift` y `push` que realizan lo inverso respectivamente. `unshift` agrega un elemento o una lista de elementos al comienzo de una lista y `push` hace lo mismo, pero al final de la lista.

```
In [112... my $var = shift @arreglo;
print dump $var;
```

```
[1]
```

Out[112]:

```
In [113... my $var1 = shift @$var;
print dump $var1;
```

```
1
```

Out[113]:

Ejemplo:

```
In [114... @A = ('a' .. 'e');
$B = shift @A; # $b se instancia con 'a'
for (my $n = 0; $n < @A; $n++) {
    print $A[$n], " ";
}
```

b c d e

```
In [115... # Se imprimen los valores b, c, d y e
@A = ('a' .. 'e');
$B = pop @A; # $b se instancia con 'e'
for (my $n = 0; $n < @A; $n++) {
    print $A[$n], " ";
}
```

a b c d

```
In [116... # Se imprimen los valores a, b, c y d
unshift @A, 1; # agrega 1 al principio del arreglo
push @A, 9; # agrega 9 al final del arreglo
for (my $n = 0; $n < @A; $n++) {
    print $A[$n], " ";
}
print "\n", "@A";
```

1 a b c d 9

1 a b c d 9

Out[116]: 1

Con las funciones descritas es posible construir una estructura de datos denominada **pila** (stack); se caracteriza por que su acceso es LIFO, (last input first output) es decir que el último elemento añadido es el primero en ser retirado, para lo cual se utiliza la función `shift` para extraer y la función `unshift` para insertar. Alternativamente se podría las funciones `push` y `pop` también pueden ser usadas para operar una pila.

Toda estructura de datos llamada **cola** opera en modo FIFO (First Input First Output), donde el primer elemento en ingresar es el primer elemento en salir. Por ejemplo, se podría implementar una cola leyendo con `shift` y agregando con `push`. Alternativamente se podría leer con la función `pop`, y agregar al inicio con `unshift`.

A continuación se muestran invertidos una lista de números ingresados

Ejemplo:

```
In [117... my @pila = (); # se crea la pila
my @datos = (2, 4, 6);
while (@datos) {
    unshift @pila, shift @datos; # se agrega un elemento a la pila
}
print "\@datos: ", dump(@datos);
print "\n\@pila: ", dump(@pila);
```

@datos: ()

@pila: (6, 4, 2)

Out[117]: 1

```
In [118... @pila = (); # se crea la pila
@datos = (2, 4, 6, -1);
while (my $numero = shift @datos) {
    unshift @pila, $numero; # se agrega un elemento a la pila
}
print "\@datos: ", dump(@datos);
print "\n\@pila: ", dump(@pila);
```

```
@datos: ()
@pila: (-1, 6, 4, 2)
```

Out[118]: 1

```
In [119... @pila = (); # se crea la pila
@datos = (2, 4, 6, -1);
while (@datos) {
    unshift @pila, shift @datos; # se agrega un elemento a la pila
}
print "\@datos: ", dump(@datos);
print "\n\@pila: ", dump(@pila);
```

```
@datos: ()
@pila: (-1, 6, 4, 2)
```

Out[119]: 1

```
In [120... my $l = @pila;
for(my $i = 0; $i < $l; $i++) {
    print shift(@pila), " "; # se extrae elemento de la pila
}
```

```
-1 6 4 2
```

```
In [121... my @enumerate = ('a' .. 'e');
while (my ($i, $valor) = each @enumerate){
    printf "i: %d value: %s\n", $i, $valor;
}
```

```
i: 0 value: a
i: 1 value: b
i: 2 value: c
i: 3 value: d
i: 4 value: e
```

Out[121]: 0

Arreglos Asociativos

Se define un arreglo asociativo con el símbolo `\%`. El carácter `\%` sirve para referirse a **todos** los miembros del arreglo asociativo. El carácter `$` se utiliza para referirse a **un solo** elemento escalar de dicho arreglo.

El siguiente código permite listar los pares clave-valor ordenados por valor

```
In [122... my %months = (Jan => 1, Feb => 2, Mar => 3,
               Apr => 4, May => 5, Jun => 6,
               Jul => 7, Aug => 8, Sep => 9,
               Oct => 10, Nov => 11, Dec => 12);
print dump keys %months;
```

```
(
    "Mar",
    "Dec",
    "Jun",
    "Nov",
    "Jul",
    "Feb",
    "Oct",
    "Aug",
    "Sep",
    "Apr",
    "May",
    "Jan",
)
```

Out[122]: 1

```
In [123... # Por defecto, un arreglo asociativo no preserva el orden de ingreso
print dump values %months;
```

```
(3, 12, 6, 11, 7, 2, 10, 8, 9, 4, 5, 1)
```

Out[123]: 1

```
In [124... foreach my $key (keys %months) { # Barrido a través de los keys
    print "$key = %months{$key}\n";
}
```

```
Mar = 3
Dec = 12
Jun = 6
Nov = 11
Jul = 7
Feb = 2
Oct = 10
Aug = 8
Sep = 9
Apr = 4
May = 5
Jan = 1
```

```
In [125... # Para preservar el orden de ingreso de un arreglo asociativo, declararlo vinculando con
```

```
tie my %months2, "Tie::IxHash";
%months2 = (Jan => 1, Feb => 2, Mar => 3,
            Apr => 4, May => 5, Jun => 6,
            Jul => 7, Aug => 8, Sep => 9,
            Oct => 10, Nov => 11, Dec => 12);
print dump keys %months2;
```

```
(
    "Jan",
    "Feb",
    "Mar",
    "Apr",
    "May",
    "Jun",
    "Jul",
    "Aug",
    "Sep",
    "Oct",
    "Nov",
    "Dec",
)
```

Out[125]: 1

```
In [126... print dump values %months2;
```

```
(1 .. 12)
```

```
Out[126]: 1
```

```
In [127... my %stock = (); # Creamos arreglo asociativo vacío  
print dump %stock;
```

```
()
```

```
Out[127]: 1
```

```
In [128... %stock = (limones => 6, peras => 3, uvas => 2); # Creamos arreglo asociativo con datos  
# Use el caracter % para referirse a todos los miembros del arreglo asociativo  
print dump %stock;
```

```
("uvas", 2, "limones", 6, "peras", 3)
```

```
Out[128]: 1
```

Para referirse a un arreglo asociativo en su totalidad, se usa su nombre precedido del símbolo %; si por el contrario, se desea acceder solamente a un elemento se usa el nombre del arreglo anteponiendo el símbolo \$.

```
In [129... # Use el caracter $ para referirse a un miembro específico del arreglo asociativo  
print $stock{peras};
```

```
3
```

```
Out[129]: 1
```

```
In [130... $stock{peras} = 5; # Cambiamos el valor de ese elemento  
print $stock{peras};
```

```
5
```

```
Out[130]: 1
```

Para asignar nuevos valores a un arreglo asociativo, se utilizan las llaves {} alrededor de la clave.

```
In [131... $stock{bananas} = 9; # Registro de bananas por primera vez  
print dump %stock;
```

```
("peras", 5, "uvas", 2, "limones", 6, "bananas", 9)
```

```
Out[131]: 1
```

```
In [132... $stock{bananas}--; # Decrementamos el valor de bananas  
print dump %stock;
```

```
("peras", 5, "uvas", 2, "limones", 6, "bananas", 8)
```

```
Out[132]: 1
```

```
In [133... $stock{bananas} -= 3; # Cambiar el valor de bananas  
print dump %stock;
```

```
("peras", 5, "uvas", 2, "limones", 6, "bananas", 5)
```

```
Out[133]: 1
```

Sobre los arreglos asociativos se se realizan tres operaciones principales:

1. obtener el valor asociado a una clave
2. cambiar el valor asociado a una clave.
3. Verificar si una clave existe.

```
In [134... if (exists $stock{bananas}){
    print "Bananas exist.\n";
}elseif(!exists $stock{bananas}){ # Equivalente a else
    print "Bananas do not exist.\n";
}

if (exists $stock{aguacates}){
    print "Aguacates exist.\n";
}else{
    print "Aguacates do not exist.\n";
}
```

Bananas exist.
Aguacates do not exist.

Out[134]: 1

```
In [135... # Para eliminar un elemento.
if (exists $stock{bananas}){
    delete $stock{bananas};
}
print dump %stock;
# Agregar aguacates:
$stock{aguacates} = 5;
print "\n", dump %stock;
```

("peras", 5, "uvas", 2, "limones", 6)
("uvas", 2, "limones", 6, "aguacates", 5, "peras", 5)

Out[135]: 1

Si lo que se quiere es manipular las llaves o los valores se usa el símbolo \@ antes del nombre.

En el siguiente ejemplo se define un arreglo asociativo llamado stock con sus pares de clave-valor.

A la estructura anterior se le pueden agregar ó modificar pares clave-valor de la siguiente forma

Una forma de crear un arreglo asociativo es a partir de pares de datos clave y valor leídos de un archivo

Ejemplo:

```
In [136... print dump [split " ", "peras", 2];
```

["peras", " 2"]

Out[136]: 1

Para ejecutar el ejemplo a continuación, crear un archivo frutas.txt y ingresar 1 fruta separada por coma seguida de la cantidad. Haga lo mismo en una nueva fila para la segunda fruta.

```
In [137... sub alimenta_hash{
  my %stock = (); # Limpieza del arreglo asociativo
  open FILE, "../data/frutas.txt" or die("Error $!");
  while (<FILE>){
    chomp($_);
    map {$stock{$_->[0]} = $_->[1]} [split ",", $_];
  }
  close FILE or die("Error $!");
  return \%stock;
}
```

```
In [138... my $stock = alimenta_hash();
print dump $stock;

{ aguacate => 3, manzana => 2 }
Out[138]: 1
```

```
In [139... print dump (keys %$stock);

("aguacate", "manzana")
Out[139]: 1
```

```
In [140... print dump (values %$stock);

(3, 2)
Out[140]: 1
```

```
In [141... foreach my $clave (keys %$stock) {
  print "$clave = $stock->{$clave}\n";
}

aguacate = 3
manzana = 2
```

```
In [142... while (my ($key, $value) = each(%$stock)){
  print $key, "\t", $value, "\n";
}

aguacate      3
manzana      2
Out[142]: 0
```

Un par clave-valor se borra con

```
In [143... delete $stock->{manzana};

Out[143]: 2
```

Un arreglo completo se borra

```
In [144... undef %$stock;
print dump $stock;

{}
```


Out[144]: 1

Para eliminar todas los pares claves-valor, pero no la estructura, se utiliza

```
In [145... %$stock = ();  
print dump $stock;
```

```
{}
```

Out[145]: 1

Para navegar una estructura se pueden utilizar varias formas. La instruccion foreach permite recorrer la estructura de forma completa. La variable \$clave se instancia con el valor de la clave.

```
In [151... $stock = alimenta_hash();  
foreach $clave (keys %$stock) {  
    printf "fruta: %s: %d\n", $clave, $stock->{$clave};  
}
```

```
fruta: aguacate: 3
```

```
fruta: manzana: 2
```

También con el elemento map, es posible listar las claves.

```
In [153... print map "$_ = $stock->{$_}\n", keys %$stock;
```

```
aguacate = 3
```

```
manzana = 2
```

Out[153]: 1

Se puede utilizar while y each para leer una arreglo asociativo

```
In [156... while (my ($key, $valor) = each %$stock){  
    print "$key = $valor\n";  
}
```

```
aguacate = 3
```

```
manzana = 2
```

Out[156]: 0

La función keys crea un arreglo indexado con las claves de un arreglo asociativo

```
In [158... my %A = ( x => 5, y => 3, z => 'abc' );  
@B = keys %A; # @b se instancia con ( 'x', 'y', 'z' );  
my @v = values %A;  
print "keys: @B\n";  
print "values: @v\n";
```

```
keys: y x z
```

```
values: 3 5 abc
```

Out[158]: 1

```
In [159... my %dict = ();
map {$_->[0]} = $_->[1]} zip (\@B, \@v);
print dump %dict;

("y", 3, "z", "abc", "x", 5)
1
Out[159]:
```

La función values devuelve un arreglo indexado con los valores de arreglo asociativo

```
In [160... %A = ( x => 5, y => 3, z => 'abc' );
@v = values %A; # @v se instancia con ( 5, 3, 'abc' );
print "@v";

abc 5 3
1
Out[160]:
```

La función exists prueba si existe una clave en un arreglo asociativo

```
In [161... %A = ( x => 5, y => 3, z => 'abc' );
$B = exists $A{z}; # $b se instancia con 1, es verdadero
$c = exists $A{w}; # $c queda con "", es falso
print dump ($B, $c);

(1, "")
1
Out[161]:
```

Es posible ordenar una estructura. Para Perl existen tres formas: por orden ASCII numericamente o alfabeticamente.

```
In [162... my %hash = (Apples => 1, apples => 4,
             artichokes => 3, Beets=> 9);
foreach my $key (sort keys %hash) {
    print "$key = $hash{$key}\n";
}

Apples = 1
Beets = 9
apples = 4
artichokes = 3
```

La salida será

```
Apples = 1
Beets = 9
apples = 4
artichokes = 3
```

Referencias a hashes

```
In [163... my $monthsref = \%months;
foreach my $key (keys %$monthsref){
    print "$key = $monthsref->{$key}\n";
}
```

```
Mar = 3
Dec = 12
Jun = 6
Nov = 11
Jul = 7
Feb = 2
Oct = 10
Aug = 8
Sep = 9
Apr = 4
May = 5
Jan = 1
```

In [164]... *# También puedo crear una referencia mediante el uso de las llaves*

```
my $meses = {Jan => 1, Feb => 2, Mar => 3,
             Apr => 4, May => 5, Jun => 6,
             Jul => 7, Aug => 8, Sep => 9,
             Oct => 10, Nov => 11, Dec => 12};
```

Out[164]: HASH(0x586fec8)

In [165]... **print** \$meses->{'Jan'};

1

Out[165]: 1

In [166]... **my** %rev_meses = reverse %\$meses; *# Intercambiamos key con values.*
print \$rev_meses{1}, "\n";
print dump %rev_meses;

```
Jan
(
  12,
  "Dec",
  7,
  "Jul",
  6,
  "Jun",
  2,
  "Feb",
  10,
  "Oct",
  9,
  "Sep",
  11,
  "Nov",
  4,
  "Apr",
  8,
  "Aug",
  5,
  "May",
  3,
  "Mar",
  1,
  "Jan",
)
```

Out[166]: 1

In [167]... **my** %hash = (Apples => [4, "Delicious red", "medium"],
 "Canadian Bacon" => [1, "package", "1/2 pound"]
);

Out[167]: ApplesARRAY(0x5857978)Canadian BaconARRAY(0x5873860)

Para extraer un valor particular, al conjunto de valores debe tratarse como un arreglo.

```
In [168... print $hash{"Canadian Bacon"}->[1];
```

package

Out[168]: 1

La instrucción anterior mostrará por pantalla el valor package que es el segundo elemento.
Es posible asignar un arreglo a un valor de clave.

```
In [169... # Agregar nueva entrada en el hash.  
$hash{"Garlic"} = [4, "cloves", "medium"];
```

Out[169]: ARRAY(0x5873a40)

```
In [170... print $hash{"Garlic"}->[1]; # Imprime cloves
```

cloves

Out[170]: 1

El siguiente código permite imprimir un hash con arreglos de distinta longitud asociados a las claves.

```
In [171... foreach my $key (keys %hash){  
    print "$key: \n";  
    foreach my $val (@{$hash{$key}}){  
        print "\t$val\n";  
    }  
    print "\n";  
}
```

Garlic:

4
cloves
medium

Apples:

4
Delicious red
medium

Canadian Bacon:

1
package
1/2 pound

```
In [172... map {print "$_: \n"; map {print "\t$_\n"} @{$hash{$_}}} keys %hash;
```

```
Garlic:
    4
    cloves
    medium
Apples:
    4
    Delicious red
    medium
Canadian Bacon:
    1
    package
    1/2 pound
```

Out[172]: 111111111

In [173]... **my** %student = ();

```
$student{'maria@epn.edu.ec'}{name} = 'Maria';
$student{'maria@epn.edu.ec'}{cedula} = 17889588534;
$student{'maria@epn.edu.ec'}{edad} = 34;
$student{'jose@epn.edu.ec'}{name} = 'Jose';
$student{'jose@epn.edu.ec'}{cedula} = 17897928798;
$student{'jose@epn.edu.ec'}{edad} = 25;

print dump %student;

(
  "maria@epn.edu.ec",
  { cedula => 17889588534, edad => 34, name => "Maria" },
  "jose@epn.edu.ec",
  { cedula => 17897928798, edad => 25, name => "Jose" },
)
```

Out[173]: 1

In [174]... **foreach my** \$id (keys %student){
 printf "correo: %s\n", \$id;
 printf "name: %s\n", \$student{\$id}{name};
 printf "cedula: %s\n", \$student{\$id}{cedula};
 printf "edad: %d\n\n", \$student{\$id}{edad};
}

```
correo: maria@epn.edu.ec
name: Maria
cedula: 17889588534
edad: 34
```

```
correo: jose@epn.edu.ec
name: Jose
cedula: 17897928798
edad: 25
```

In [175]... *# Definimos una referencia con la misma estructura de datos*

```
my $student = {};

$student->{'maria@epn.edu.ec'}{name} = 'Maria';
$student->{'maria@epn.edu.ec'}{cedula} = 17889588534;
$student->{'maria@epn.edu.ec'}{edad} = 34;
$student->{'jose@epn.edu.ec'}{name} = 'Jose';
$student->{'jose@epn.edu.ec'}{cedula} = 17897928798;
$student->{'jose@epn.edu.ec'}{edad} = 25;

print dump $student;
```

```
{
  "jose\@epn.edu.ec" => { cedula => 17897928798, edad => 25, name => "Jose" },
  "maria\@epn.edu.ec" => { cedula => 17889588534, edad => 34, name => "Maria" },
}
```

Out[175]: 1

```
In [176... my $estudiante = {
  "jose\@epn.edu.ec" => { cedula => 17897928798, edad => 25, name => "Jose" },
  "maria\@epn.edu.ec" => { cedula => 17889588534, edad => 34, name => "Maria" },
};

foreach my $id (keys %$estudiante){
  print "$id\n";
  print $estudiante->{$id}{name}, "\n";
  print $estudiante->{$id}{cedula}, "\n";
  print $estudiante->{$id}{edad}, "\n\n";
}
```

```
maria@epn.edu.ec
Maria
17889588534
34
```

```
jose@epn.edu.ec
Jose
17897928798
25
```

```
In [177... # Operación Not in set:
my %ignore = map {$_ => 1} ('c', 'ignore');
print dump (keys %ignore);
```

```
("c", "ignore")
```

Out[177]: 1

```
In [178... print dump grep {!exists $ignore{$_}} ('a', 'b', 'c', 'ignore');
```

```
("a", "b")
```

Out[178]: 1

Operadores

"No puedo decir que no estoy en desacuerdo contigo". Groucho Marx

En Perl los valores falsos son:

En cadenas "", '', "0"

El número 0.

El valor "no definido", cuando una variable existe, pero no tiene valor asociado

Una lista vacía también es falso.

Todos los demás valores son verdaderos

Los operadores clásicos son:

|Operación |Para aritmética |Para cadenas| |-----|:-----|:-----:| | Mayor que | > | gt (greater than)
| | Menor que | < | lt (less than) | | Mayor o igual | >= | ge (greater equal)| | Menor o igual | <= | le (less
equal)| | Igual | == | eq (equals)| | Diferente != | ne (not equal)| | Compara | <=> | cmp (compare)|

El operador compara retorna un valor de -1, 0 o 1 dependiendo si el valor a la izquierda es menor, igual o mayor que el de la derecha del operador.

Algunos operadores lógicos que permiten determinar el valor de verdad de una expresión son:

Operador	Función
!	Negación Lógica
-	Inverso aditivo (cambia el signo de un numero)
~	Negación Bit a bit de un valor
&&	AND - Conjunción lógica de los operandos
	OR - Disyunción lógica de los operandos
&	AND - Conjunción bit a bit de los operandos
	OR - Disyunción bit a bit de los operandos
^	XOR - Or exclusiva bit a bit
..	Si evaluado como escalar regresa alternativamente verdadero y falso. Como arreglo genera un arreglo a partir de los límites de un rango
not	Negación lógica de la expresión a su izquierda (sinónimo de baja prioridad)
and	Conjunción lógica (sinónimo de baja prioridad)
or	Disyunción (sinónimo de baja prioridad)
xor	Disyunción exclusiva lógica (sinónimo de baja prioridad))

Ejemplo:

In [179]...

```
$A = 0;  
$B = 1;  
  
print "A y B resulta verdadero\n" if $A and $B;  
print "A o B resulta verdadero\n" if $A or $B;  
print "A xor B resulta verdadero\n" if $A xor $B;  
print "A nand B resulta verdadero\n" if not ($A and $B);
```

```
A o B resulta verdadero  
A xor B resulta verdadero  
A nand B resulta verdadero
```

Out[179]: 1

Existen los operadores && y || (como en el lenguaje C) y poseen mas prioridad que and y or.

comparación de números

In [180]...

```
my $x = $A <=> $B;  
# $x queda con -1 si $A < $B  
# $x queda con 0 si $A == $B  
# $x queda con 1 si $A > $B
```

Out[180]: -1

Comparación de strings

```
In [181... $x = 'aba' cmp 'abc';  
# $x queda con -1 si $A lt $B  
# $x queda con 0 si $A eq $B  
# $x queda con 1 si $A gt $B
```

Out[181]: -1

Así como los operadores aritméticos fuerzan a la variable a comportarse como de tipo entero o real, los operadores de cadena fuerzan a los escalares a comportarse como cadenas, de modo que los números son automáticamente convertidos en cadenas al aplicarles estos operadores.

Si una variable está indefinida, al ser evaluada da una cadena vacía, lo cual significa falso. Para definirla, se puede aplicar la siguiente técnica:

```
In [182... my $costo;  
$costo = 100 unless $costo; # Si costo no esta definido, inicializarlo con 100  
# unless es lo mismo que 'if not'
```

Out[182]: 100

La función `defined` se utiliza para averiguar si una variable está definida.

Una variable definida significa que hay un valor asignado.

```
In [183... $A = 5;  
$B = undef;  
print "variable \"$B\" definida con el valor $A.\n" if defined $A;  
print "variable \"$B\" no está definida.\n" if !defined $B;  
$costo = 100 unless defined ($costo); # Acuerdese que lo que está a la izquierda  
# de unless se ejecuta si
```

variable \$B definida con el valor 5.
variable \$B no está definida.

Out[183]: 1

```
In [184... %hash = ();  
if (%hash){  
    print "Variable \"%hash\" definida.\n";  
}
```

Out[184]: 0

```
In [185... my @array = ();  
if (@array){  
    print "Variable \@array definida.\n";  
}
```

Out[185]: 0

Nota: la función `undef` lo que hace es desligar un valor de una variable.

Las operaciones a realizar con cadenas, por lo regular se manejan como funciones en vez de como operadores, con la excepción de las siguientes:

Operador Función

.	Concatenación, todas las cadenas operadas por "." forman una sola cadena. \$a = "Hola"; \$b = "mundo"; \$c = \$a . " " . \$b;
..	Operador que permite definir rangos. Por ejemplo: @A = "ay" .. "bb"; @A queda con ("ay", "az", "ba", "bb") @A = 1 .. 5; @a queda con (1, 2, 3, 4, 5)
x	Operador de repetición, en contexto escalar toma cadena de la izquierda y la repite el numero de veces que el numero de la derecha indica. En contexto de una lista, toma la lista a su izquierda y la repite las veces que indica el operador a su derecha en una nueva lista. bar = '-' x 72; # devuelve una cadena de 72 guiones.
++	Autoincremento, modifica el valor de los elementos de una cadena de modo que se incrementen sin cambiar su naturaleza. Por ejemplo ++"zzz" da "aaaa"; ++"99" da "100" , ++"a9" da "b0".

```
In [186... my $c = $A . " " . "cadena";
```

```
Out[186]: 5 cadena
```

```
In [187... print dump (1 .. 5);
```

```
(1 .. 5)
1
Out[187]:
```

```
In [188... print '-' x 5;
```

```
-----
1
Out[188]:
```

```
In [189... @array = ('word') x 5;
print dump @array;
```

```
("word", "word", "word", "word", "word")
1
Out[189]:
```

```
In [190... print $A, "\n";
$A++;
print $A, "\n";
print $A++, "\n";
print $A, "\n";
```

```
5
6
6
7
1
Out[190]:
```

Estructuras de Control

¿Por qué las cosas son como son y no de otra manera?" Blaise Pascal

Las estructuras principales de Perl y su sintaxis son:

IF:

```
if (EXPRESIÓN) BLOQUE  
if (EXPRESIÓN) BLOQUE else BLOQUE  
if (EXPRESIÓN) BLOQUE elsif (EXPRESIÓN) BLOQUE ... else BLOQUE
```

WHILE:

```
while (EXPRESIÓN) BLOQUE  
  
do BLOQUE until | while (EXPRESIÓN)
```

FOR:

```
for (EXPRESION;EXPRESION;EXPRESION) BLOQUE  
  
BLOQUE for @lista;
```

FOREACH:

```
foreach VARIABLE (LISTA) BLOQUE
```

Donde:

BLOQUE es un conjunto de comandos agrupados por los caracteres {}
EXPRESION es una proposición lógica que evaluada devuelve un valor de verdad.

LISTA es un conjunto de valores separados por coma.

Ejemplo if

In [191]...

```
my $objeto = "tiza";  
my $requerido = "tiza";  
my $cantidad = 22;  
  
if ($objeto eq "tiza") {  
    print "la variable objeto esta instanciada con tiza\n";  
} else {  
    print "la variable objeto NO esta instanciada con tiza\n";  
}  
  
if ($cantidad <= 25) {  
    print "Hay que reponer " . ((25 - $cantidad) + 3) . " elementos\n";  
}
```

la variable objeto esta instanciada con tiza
Hay que reponer 6 elementos

Out[191]: 1

Ejemplo if-elsif-else

Si la primer expresión es verdadera ejecutar bloque, de lo contrario evaluar las expresiones elsif, si alguna es verdadera ejecutar bloque correspondiente, si finalmente se obtiene falso ejecutar bloque definido en else

Ejemplo:

In [192]...

```
my ($dividendo, $divisor, $resultado) = (4, 2);
```

```

if ($divisor == 0) {
    print STDERR "Error: no puedo dividir por cero!\n";
}
elseif ($dividendo == 0){
    $resultado = $dividendo;
}
elseif ($divisor == 1){
    $resultado = $dividendo;
}
else {
    $resultado = $dividendo / $divisor;
}

print "El resultado es ", $resultado, "\n" if $divisor != 0;

```

El resultado es 2

Out[192]: 1

Ejemplo unless

La estructura unless, equivale a la negación de if, es decir que si la condición de unless da por resultado falso se ejecutan las instrucciones del bloque.

```

In [193... my $text = "Ingrese el nombre de un sistema operativo: ";
print $text;
my $nombre = "LINUX";

#if(lc($nombre) eq "linux"){
if($nombre =~ m/linux/i){
    print "Ese sí es un buen producto.\n";
}
else{
    print "Le pedi que ingresara un nombre de sistema operativo.\n";
}

```

Ingrese el nombre de un sistema operativo: Ese sí es un buen producto.

Out[193]: 1

```

In [194... while ($text =~ m/\b([a-z+)\b/g){
    printf "%s\n", $1;
}

```

el
nombre
de
un
sistema
operativo

```

In [195... print "Escriba un número mayor a 10.\n";
my $numero = 10;
unless ($numero > 10) { # If not
    print "Error: $numero no es mayor a 10.\n";
}else{
    print "Número $numero es mayor a 10.\n";
}

```

Escriba un número mayor a 10.
Error: 10 no es mayor a 10.

Out[195]: 1

Ejemplo while

Ejecutar bloque siempre que el resultado de la evaluación de la expresión sea verdadero

```
In [196... my ($i, $suma) = (0, 0);
while ($i < 10) {
    $suma = $suma + $i++;
}
print "El resultado de la suma es: $suma";
```

El resultado de la suma es: 45

Out[196]: 1

Ejemplo do while

```
In [197... ($i, $suma) = (0, 0);

do{
    $suma = $suma + $i++;
}while($i < 10);

print "El resultado de la suma es: $suma";
```

El resultado de la suma es: 45

Out[197]: 1

Ejemplo until

```
In [198... ($i, $suma) = (0, 0);

until($i == 10) { # mientras sea falso = hasta que sea verdad
    $suma = $suma + $i++;
}
print "El resultado de la suma es: $suma.\n";
print "El valor de i es: $i.\n";
```

El resultado de la suma es: 45.

El valor de i es: 10.

Out[198]: 1

```
In [199... ($i, $suma) = (0, 0);

until($i >= 10) { # Hasta que sea verdadero
    $suma = $suma + $i;
    $i++;
}
print "El resultado de la suma es: $suma.\n";
print "El valor de i es: $i.\n";
```

El resultado de la suma es: 45.

El valor de i es: 10.

Out[199]: 1

Ejemplo do until:

```
In [200... ($i, $suma) = (0, 0);

do{
    $suma = $suma + $i++;
}while($i == 10);
```

```
print "El resultado de la suma es: $suma.\n";
print "El valor de i es: $i.\n";
```

El resultado de la suma es: 45.
El valor de i es: 10.

Out[200]: 1

Ejemplo for

Itere mientras la condición no se cumpla.

```
In [201... $suma = 0;

for(my $i = 0; $i < 10; $i++) {
    $suma = $suma + $i;
}
print "El resultado de la suma es $suma \n";
```

El resultado de la suma es 45

Out[201]: 1

```
In [202... for(my ($i, $j) = (0, 0); $i < 10; $i++, $j+=2) {
    printf "i:%d j:%d\n", $i, $j;
}
```

i:0 j:0
i:1 j:2
i:2 j:4
i:3 j:6
i:4 j:8
i:5 j:10
i:6 j:12
i:7 j:14
i:8 j:16
i:9 j:18

Ejemplo foreach

Itera la cantidad de elementos que haya en la lista de datos, en cada vuelta toma el enésimo elemento y lo asigna a una variable.

```
In [203... @lista = (7, 11, 22, 5, 6, 7, 45);
foreach my $elemento (@lista) {
    printf "%d ", $elemento;
}
```

7 11 22 5 6 7 45

```
In [204... $text = "cadena";
print join '- ', ($text =~ m/([aeiou]+[aeiou]+)/g);
```

ca- de- na

Out[204]: 1

```
In [205... $text = "cadena";
foreach my $syl ($text =~ m/([aeiou]+[aeiou])/g){
    printf "%s\n", $syl;
}
```

ca
de
na

```
In [206... foreach my $elemento (5 .. 9) {  
    print $elemento . " ";  
}
```

5 6 7 8 9

Ejemplo for de una fila

```
In [207... print $_ . " " for @lista;
```

7 11 22 5 6 7 45

Ejemplo last y next

En Perl, la instrucción `last` se utiliza para forzar la finalización de un ciclo `while`. En cambio la instrucción `next` fuerza inmediatamente a la evaluación de la condición, saltando las siguientes instrucciones del bloque.

```
In [208... foreach my $elemento (1 .. 10) {  
    print $elemento . " ";  
    last if $elemento == 5; # Abandona el bucle  
}
```

1 2 3 4 5

```
In [209... foreach my $elemento (1 .. 10) {  
    next if $elemento == 5; # Saltar la ejecucion 1 vez  
    print $elemento . " ";  
}
```

1 2 3 4 6 7 8 9 10

Funciones y subrutinas

No creo en una vida posterior, pero por si acaso me he cambiado de ropa interior". Woody Allen

Perl no hace diferencias entre funciones y subrutinas. Una función se define:

```
In [210... sub misubrutina{  
    print "Soy una misubrutina.\n";  
}
```

y se la invoca:

```
In [211... misubrutina();
```

Soy una misubrutina.

Out[211]: 1

Si se quieren enviar y recibir parámetros:

```
In [212... sub maximo {  
    my ($var1, $var2) = @_; # @_ contiene los parámetros  
    return $var1 > $var2 ? $var1 : $var2;  
}
```

```
In [213... sub maximo{  
    return (sort {$b <=> $a} @_)[0]; # @_ contiene los parámetros
```

```
}
```

Subroutine `maximo` redefined at reply input line 1.

```
In [214... print maximo(22, 55, 34, 75);
```

75

```
Out[214]: 1
```

Por defecto, si no se especifica el valor a devolver con `return`, el valor retornado por la función es el último valor evaluado dentro de su bloque.

```
In [215... sub saludar{
  print "¡Hola $_[0]!\n";
}
```

```
saludar("Cesar");
saludar("Sandra");
```

```
¡Hola Cesar!
¡Hola Sandra!
```

```
Out[215]: 1
```

```
In [216... sub function{
  my %args = @_;
  foreach my $key (keys %args){
    print $key, " ", $args{$key}, "\n";
  }
}
```

```
In [217... function('var1' => 5, 'var2' => 2);
```

```
var2 2
var1 5
```

```
In [218... sub function2{
  my %args = @_;

  print "Nombre: ", $args{nombre}, "\n";
  print "Edad: ", $args{edad}, "\n";
}
```

```
In [219... function2('nombre' => 'Maria', 'edad' => 20);
```

```
Nombre: Maria
Edad: 20
```

```
Out[219]: 1
```

```
In [220... sub function3{
  my %args = (enfermo => 0, @_); # valor por defecto
                                # enfermo es opcional

  if (!defined $args{nombre}){
    print STDERR "Argument Nombre faltante.\n";
    return;
  }

  if (!defined $args{edad}){
    print STDERR "Argument Edad faltante.\n";
    return;
  }
}
```

```

print "Nombre: ", $args{nombre}, "\n";
print "Edad: ", $args{edad}, "\n";
print "Enfermo: ", $args{enfermo}, "\n";
}

```

In [221... `function3('nombre' => 'Maria', 'edad' => 20);`

```

Nombre: Maria
Edad: 20
Enfermo: 0

```

Out[221]: 1

In [222... `function3('nombre' => 'Mario');`

Argument Edad faltante.

In [223... `function3('nombre' => 'Maria', 'edad' => 20, enfermo => 1);`

```

Nombre: Maria
Edad: 20
Enfermo: 1

```

Out[223]: 1

In [224... `my @personas = (['nombre' => 'Maria', 'edad' => 20],
['nombre' => 'Jose', 'edad' => 30]);`

```

foreach my $person (@personas){ # foreach
    function2(@$person);
}

```

```

Nombre: Maria
Edad: 20
Nombre: Jose
Edad: 30

```

In [225... `sub obtiene_frutas{
 my @stock = (); # Limpieza del arreglo indexado
 open FILE, "data/frutas.txt" or die("Error $!");
 while (<FILE>){
 chomp($_);
 push @stock, map {['fruta' => $_->[0], 'cantidad' => $_->[1]]} [split " ", $_];
 }
 close FILE or die("Error $!");
 return @stock;
}`

In [226... `sub function4{
 my %args = @_;

 print "Fruta: ", $args{fruta}, " ";
 print "Cantidad: ", $args{cantidad}, "\n";
}`

In [227... `my @stock = obtiene_frutas();

foreach my $fruta (@stock){ # foreach
 function4(@$fruta);
}`

Error No such file or directory at reply input line 3.

In [228... `print dump @stock;`


```
( )  
Out[228]: 1
```

El pasaje de parámetros, realizado en el siguiente ejemplo es por referencia, es decir que se envían un punteros indicando la dirección de memoria donde están los datos. Si dentro de la función se modificara algún valor sobre los parámetros pasados, se modificarían las variables originales. Luego de ejecutar el siguiente programa, muestre los valores de *a*, *b* y saque sus propias conclusiones.

Variables locales

Una variable declarada con "my" es local en el ámbito de la subrutina. Se aconseja la práctica de copiar los argumentos a variables locales. Con esto se consiguen dos efectos: darles nombres más significativos, y hacer que el traspaso sea por valor (el default es por referencia).

```
In [229... sub suma{  
    my($A, $B) = @_; # @_ es la lista de los parámetros recibidos  
  
    $A + $B; # se podría haber devuelto el resultado  
} # con la función return  
  
print suma(2, 3), "\n";
```

```
5  
Out[229]: 1
```

Si se tiene un número variable de parámetros se los podría recibir en una lista local

```
In [230... print suma(3, 35, 22, 5), "\n";
```

```
38  
Out[230]: 1
```

```
In [231... sub suma2{  
    my $s = 0;  
  
    foreach my $x (@_) {  
        if ($x !~ m/^-?\d+(?:\.\d+)?$/){  
            print STDERR "Valor '$x' no es un número válido.\n";  
            return;  
        }  
        $s += $x;  
    }  
    return $s;  
}
```

```
In [232... print suma2( 1 .. 8, "xa" ), "\n";
```

```
Valor 'xa' no es un número válido.  
Out[232]: 1
```

Orientación a Objetos

```
In [233... # Una clase tiene atributos (propiedades) y métodos (funciones).  
  
# Se define una clase con la palabra package
```

```
# Una clase que tenga un constructor (new) permite crear un objeto.  
  
# Una clase sin constructor se llama clase estática.
```

```
In [234... package Empleado{  
    use strict;  
    use warnings;  
  
    sub new{  
        my ($class, $nombre, $email) = (shift, @_);  
  
        my $self = {nombre => $nombre,  
                    email => $email,  
                    };  
        return bless ($self, $class);  
    }  
  
    sub nombre{  
        my ($self, $nombre) = @_;  
        $self->{nombre} = $nombre if $nombre;  
        return $self->{nombre};  
    }  
  
    sub email{  
        my ($self, $email) = @_;  
        $self->{email} = $email if $email;  
        return $self->{email};  
    }  
  
    1;  
}
```

```
Out[234]: 1
```

```
In [235... my $empleado = new Empleado('Maria', 'maria@epn.edu.ec');
```

```
Out[235]: Empleado=HASH(0x58b5668)
```

```
In [236... print dump $empleado;
```

```
bless({ email => "maria\@epn.edu.ec", nombre => "Maria" }, "Empleado")  
Out[236]: 1
```

```
In [237... print $empleado->nombre();
```

```
Maria  
Out[237]: 1
```

```
In [238... print $empleado->email();
```

```
maria@epn.edu.ec  
Out[238]: 1
```

```
In [239... my $empleado2 = new Empleado();
```

```
Out[239]: Empleado=HASH(0x58c1900)
```

```
In [240... print dump $empleado2;
```

```
bless({ email => undef, nombre => undef }, "Empleado")
```

```
Out[240]: 1
```

```
In [241... $empleado2->nombre('Jose');  
$empleado2->email('jose@epn.edu.ec');
```

```
Out[241]: jose@epn.edu.ec
```

```
In [242... print dump $empleado2;
```

```
bless({ email => "jose@epn.edu.ec", nombre => "Jose" }, "Empleado")
```

```
Out[242]: 1
```

Herencia

Decimos que una clase hereda los atributos y métodos de una clase padre cuando pasa a visualizar y tener acceso a los mismos a partir de la clase hija.

Una forma de hacer eso, es mediante la instrucción:

```
use base qw(Clase_Padre);
```

Para acceder sus métodos internamente se lo hace mediante la palabra SUPER, por ejemplo:

```
$self->SUPER::metod()
```

Externamente se lo hace directamente a través del objeto que instanciado a partir de la clase hija, como si fuera su propio método o propiedad.

```
In [243... package Sueldo{  
    use strict;  
    use warnings;  
    use base qw(Empleado);  
  
    sub new{  
        my ($class, $dias_trabajados, $sueldo_diario) = (shift, @_);  
  
        my $self = {  
            dias_trabajados => $dias_trabajados // 0,  
            sueldo_diario   => $sueldo_diario // 0,  
        };  
        return bless ($self, $class);  
    }  
  
    sub sueldo{  
        my $self = shift @_;  
  
        # print "Father class: ", $self->SUPER::nombre(), "\n";  
  
        return $self->{dias_trabajados} * $self->{sueldo_diario};  
    }  
}
```

```
In [244... my $sueldo = new Sueldo();
```

```
Out[244]: Sueldo=HASH(0x58c5088)
```

```
In [245... print dump $sueldo;
```

```
    bless({ dias_trabajados => 0, sueldo_diario => 0 }, "Sueldo")
```

Out[245]: 1

```
In [246... $sueldo->nombre('Brian');  
$sueldo->email('brian@epn.edu.ec');
```

Out[246]: brian@epn.edu.ec

```
In [247... $sueldo->{dias_trabajados} = 22;  
$sueldo->{sueldo_diario} = 20;
```

Out[247]: 20

```
In [248... print dump $sueldo;
```

```
    bless({  
      dias_trabajados => 22,  
      email => "brian\@epn.edu.ec",  
      nombre => "Brian",  
      sueldo_diario => 20,  
    }, "Sueldo")
```

Out[248]: 1

```
In [249... print $sueldo->sueldo();
```

440

Out[249]: 1

```
In [250... my $sueldo2 = new Sueldo(18, 90);
```

Out[250]: Sueldo=HASH(0x58abdd8)

```
In [251... $sueldo2->nombre('Juan');  
$sueldo2->email('juan@epn.edu.ec');
```

Out[251]: juan@epn.edu.ec

```
In [252... print $sueldo2->sueldo();
```

1620

Out[252]: 1

```
In [253... print dump $sueldo2;
```

```
    bless({  
      dias_trabajados => 18,  
      email => "juan\@epn.edu.ec",  
      nombre => "Juan",  
      sueldo_diario => 90,  
    }, "Sueldo")
```

Out[253]: 1